



HIGH PERFORMANCE COMPUTING 101

HPC Storage @ IU (Module 3)

Module 3 — Lesson 1

File System Basics



Overview

- The Linux file system
- Working with files
- File system internals
- The file system tree in practice



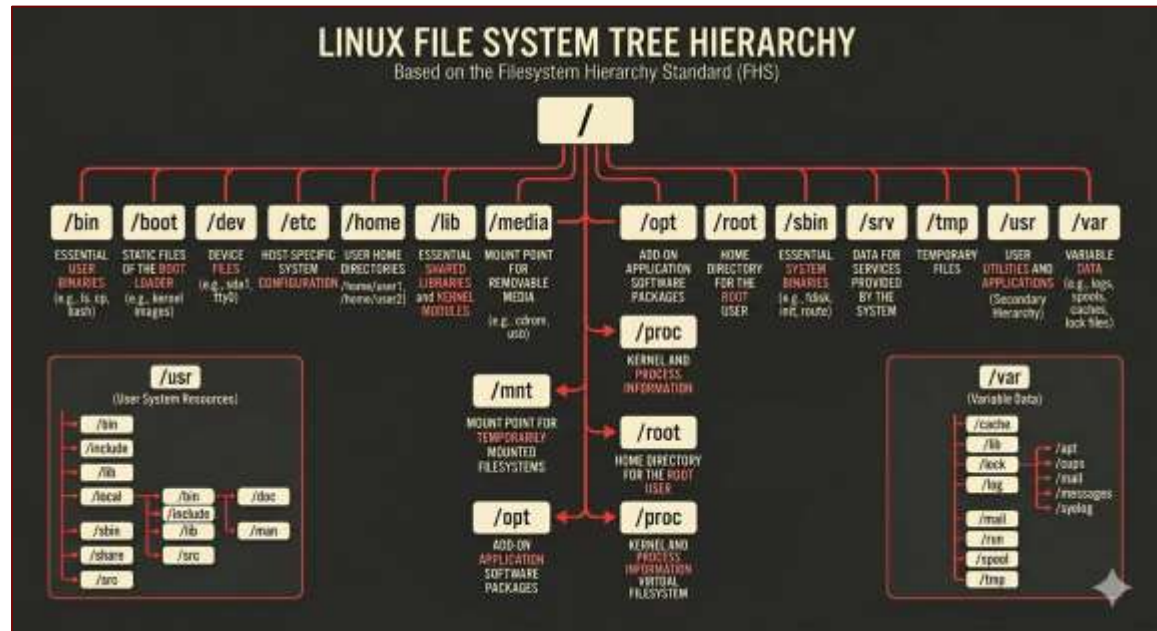
The Linux file system

- The Linux file system
- Working with files
- File system internals
- The file system tree in practice



The Linux file system

- A traditional Unix fs
- Tree-based structure
- Uses forward slash /
- / is the top-level dir
- All storage spaces are mounted as a directory in this tree
 - e.g /home might be a second hard drive





The Linux file system

- What's familiar
 - Interactions will be similar to MacOS or Windows
 - It's mostly folders (directories) and files
- What might not be familiar
 - Linux (as all Unix OS types) treats most everything as a file
 - e.g. device drivers live as files in /dev; kernel variables in /sys
 - as a user, you don't need to worry about this
- Worth noting that some special files can be helpful: e.g. /dev/null



File Interaction

- As a reminder, you can work with files and directories via both
 - the Caja GUI in RED
 - the Bash command line in any terminal window
- File interactions are not unique to the storage you work with
 - e.g. read, write, append, copy, move, remove (unlink)
 - a write to a file on a local disk is the same as to a Lustre system
 - it's just "folders and files"
- But there are things to be aware of that impact your experience



Working with files

- The Linux file system
- Working with files
- File system internals
- The file system tree in practice



Viewing files

- `$ cat <file>` — prints the entire file to the screen
- `$ less <file>` — pages through a file; press ``q`` to quit
- `$ head -n 20 <file>` — show the first 20 lines
- `$ tail -n 20 <file>` — show the last 20 lines
- `$ tail -f <file>` — follow a file as it grows (useful for watching live job output)



Creating, editing, and removing files

- `$ touch <file>` — create an empty file or update its timestamp
- `$ mkdir -p <file>` — create a directory, including any missing parent directories
- `$ cp <src> <dest>` — copy a file; adding the `-r` flag copies a directory recursively
- `$ mv <src> <dest>` — move or rename a file
- `$ rm <file>` — delete a file; adding the `-r` flag deletes a directory recursively
- Text editors available on HPC Systems
 - ``nano`` is beginner-friendly
 - ``vim`` and ``emacs`` are powerful, with steeper learning curves



File permissions and ownership

- `$ ls -l` — show a mode string and the owner for each file
 - e.g. `-rw-r--r--`
 - the mode is read as owner, group, other, each with read (r), write (w), and execute (x)
- `$ chmod` — set permissions
 - octal: ``644` = `rw-r--r--`, `755` = `rwxr-xr-x``
 - `$ chmod +x <file>` adds execute permissions
- `$ chown user:group <file>` — change owner and group
- `$ stat <file>` — show full file metadata



Finding files

- `$ find . -name "*.txt"` — find files by name starting in the current directory
- `$ find . -size + 1G` — find files larger than 1 GiB
- `$ find . -newer <ref>` — find files newer than a reference file
- `$ find . -type f` — restrict results to regular files



File system internals

- The Linux file system
- Working with files
- File system internals
- The file system tree in practice



Some details

- A few details about all file systems will improve your experience
- There is the file system structure you see
 - also structure connected to the storage platform (invisible to you)
 - there are many file system types (w/ different benefits/drawbacks)
- To create the structure you see, most file systems use
 - superblocks: file system metadata: layout, free block map, etc.
 - inodes: file metadata; every file is an inode plus payload data
 - directories: files containing name mapping to inodes



Some details

- inodes contain all file metadata, *except* filename(s)
 - seems counterintuitive
 - allows efficient storage and management
- humans (and their codes/scripts) reference files by name
 - computers use inodes
 - directory files are mappings of file name to inode
 - every directory entry for an inode is a *link* ("hard link") to inode
 - same inode/file can have multiple names, dir entries, links
 - when file name is deleted, that link is removed
 - Trivia: Unix calls file removal "unlinking"



Some details

- An inode has a link counter variable
 - when the counter goes to zero, the file is lost
 - system can reuse that storage
- The inode maps storage location(s) for chunks of data
 - yes, your file data is fragmented all over the file system
 - for reasons of efficiency
- **Key point:**
 - metadata (ownership, permissions, dates, etc) separate from data



Happy Users Limit Metadata Retrieval

- On a single disk or local storage device, metadata retrieval is simple, low-cost
- On cluster file systems, metadata usually stored separate from data
 - might be one metadata storage server for many data servers
 - metadata retrieval can be a choke point or bottleneck
- Most users have an alias for the `ls` command to add decorative info that shows directories, executable files, special files, etc
 - e.g. `ls -F` or `ls --color` have to read metadata for all files in dir



Happy Users Limit Metadata Retrieval

- The same file system that might give you 80 GB/s of file I/O might make you wait 30 seconds to list a directory of thousands of files
 - if you accidentally request the metadata lookup
- That 30 seconds will feel much longer than it is
 - recommendation is to change aliases or remove them
 - use the undecorated (no options) `ls` command to simply list files
- This takes some practice for Linux enthusiasts
 - we get used to the extra feedback in file listings
 - can be hard to undo habits



File system internals

- The Linux file system
- Working with files
- File system internals
- The file system tree in practice



A look at the file system tree

- A useful command to see the connection of storage devices to "mount points" in filesystem tree is *df*
 - recommended flags are -T and -h
 - -T shows file system type
 - -h shows "human readable" output; scales units appropriately
 - *df* shows
 - file system storage location (RAM, drive, network)
 - file system type (with the -T flag)
 - storage size, amount used, amount free & use as a percentage
 - mount point into the filesystem tree



A look at the file system tree

- Example from a personal workstation with multiple drives

```
> df -Th
```

Filesystem	Type	Size	Used	Avail	Use%	Mounted on
tmpfs	tmpfs	3.2G	2.0M	3.2G	1%	/run
tmpfs	tmpfs	16G	197M	16G	2%	/dev/shm
tmpfs	tmpfs	5.0M	4.0K	5.0M	1%	/run/lock
tmpfs	tmpfs	16G	216K	16G	1%	/run/qemu
/dev/sda4	ext4	93G	46G	42G	53%	/
/dev/sda2	ext4	1.9G	1.2G	572M	68%	/boot
/dev/sde1	xfs	477G	59G	419G	13%	/opt
/dev/md0	xfs	1.9T	173G	1.7T	10%	/home
/dev/sdd1	xfs	1.9T	284G	1.6T	16%	/data

- tmpfs is a RAM-disk: fs structure in memory for specific software use
- ext4 and xfs are common filesystem types in Linux for small-to-medium storage devices or arrays of devices
- Note: sda, sdd and sde are drives; md0 is an array of 2 mirrored drives



A look at the file system tree

- Example from Quartz (slightly edited to fit on slide)

```
> df -Thx tmpfs
```

/dev/md1	ext4	38G	75M	36G	1%	/var/mmfs
/dev/md0	ext4	1.7T	449M	1.6T	1%	/tmp
g2-home	gpfs	609T	421T	188T	70%	/geode2/home
g2-logs	gpfs	50T	37T	14T	74%	/geode2/logs
g2-projects	gpfs	1.4P	910T	443T	68%	/geode2/projects
g2-soft	gpfs	19T	16T	3.3T	83%	/geode2/soft
geode-home.rs.iu.edu:/gpfs/homedirs	nfs4	1.2P	49T	1.1P	5%	/geode3/home
geode-project.rs.iu.edu:/	nfs4	3.1P	1002T	2.1P	33%	/geode3/projects
10.10.20.10@o2ib:10.10....:/project	lustre	19P	11P	7.4P	60%	/N/project
10.10.20.10@o2ib:10.10....:/slate	lustre	1.3P	567T	734T	44%	/N/slate
10.10.20.10@o2ib:10.10....:/scratch	lustre	3.5P	1.4P	2.1P	40%	/N/scratch

- Note: "-x tmpfs" option removes all tmpfs entries from the output
- The "-t lustre" option would show *only* the entries for Lustre mountpoints
- The options can be given together or separately with a "-" on each



A look at the file system tree

- Pay close attention to these entries

```
> df -Thx tmpfs
```

/dev/md1	ext4	38G	75M	36G	1%	/var/mmfs
/dev/md0	ext4	1.7T	449M	1.6T	1%	/tmp
g2-home	gpfs	609T	421T	188T	70%	/geode2/home
g2-logs	gpfs	50T	37T	14T	74%	/geode2/logs
g2-projects	gpfs	1.4P	910T	443T	68%	/geode2/projects
g2-soft	gpfs	19T	16T	3.3T	83%	/geode2/soft
geode-home.rs.iu.edu:/gpfs/homedirs	nfs4	1.2P	49T	1.1P	5%	/geode3/home
geode-project.rs.iu.edu:/	nfs4	3.1P	1002T	2.1P	33%	/geode3/projects
10.10.20.10@o2ib:10.10....:/project	lustre	19P	11P	7.4P	60%	/N/project
10.10.20.10@o2ib:10.10....:/slate	lustre	1.3P	567T	734T	44%	/N/slate
10.10.20.10@o2ib:10.10....:/scratch	lustre	3.5P	1.4P	2.1P	40%	/N/scratch

- Geode 2 is the current IBM Storage Scale (GPFS) system
- Geode 3 is new VAST-based storage (mounted with NFS)
- Slate, Slate Project and Slate Scratch are the Lustre file systems



Networked file systems

- There are many filesystem types that can be mounted by Linux hosts
 - Lingo
 - the file system is "exported" by filesystem server
 - the file system is "mounted" by client (Quartz nodes are clients)
- The *df* syntax for the filesystem source (hostname/IP) varies with filesystem type
 - Generally has enough information for client to map to server
 - That mapping can be multi-layer (e.g. Lustre has routers)
 - Don't worry about that column, especially for networked systems



Reflection

- You now understand:
 - the Linux file system
 - working with files
 - file system internals
 - the file system tree in practice:Next: M3L2 covers IU's HPC file systems, home directories, and storage strategy
- Complete the reflection for this lesson before continuing